

```

        self.playingState = STATE_GAME_OVER
    ❹ self.draw() # force drawing of game over message

    ❺ if self.score > self.lowestHighScore:
        scoreAsString = 'Your score: ' + str(self.score) + '\n'
        if self.score > self.highestHighScore:
            dialogText = (scoreString +
                          'is a new high score, CONGRATULATIONS!')
        else:
            dialogText = (scoreString +
                          'gets you on the high scores list.')

        result = showCustomYesNoDialog(self.window, dialogText)
        if result: # navigate
            self.goToScene(SCENE_HIGH_SCORES, self.score)

    self.startButton.enable()
    self.highScoresButton.enable()
    self.soundCheckBox.enable()
    self.quitButton.enable()

```

Listing 16-6: The `update()` method of the `ScenePlay` class

The scene manager calls the `update()` method of the `ScenePlay` class ❶ in every frame. This method handles everything that happens while the game is being played. First, it tells the `Player` object to move the `Player`'s icon to the position of the mouse. Then it calls the `Player`'s `update()` method ❷, which returns the current rectangle of the icon in the window. We use this to see if the `Player`'s icon has contacted any `Goodies` or `Baddies`.

Next, it calls the `Goodie` manager's `update()` method ❸ to move all the `Goodies`. This method returns the number of `Goodies` that the `Player` has contacted, which we use to increase the score.

This is followed by a call to the `Baddie` manager's `update()` method ❹ to move all the `Baddies`. That method returns the number of `Baddies` that have fallen off the bottom of the game area.

We then check to see if the `Player` has contacted any `Baddies` ❺. If so, the game is over and we display a `Game Over` graphic. We also make a special call to our `draw()` method ❻ because we may put up a dialog for the